Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Суховило В. С.

Проверил:

Никитин С. К.

Брест 2026

Цель и задачи лабораторной работы

Цель работы: построить многослойный персептрон p–N–1 для прогноза искусственного временного ряда, сформировать выборки методом скользящего окна и экспериментально оценить влияние окна p и числа скрытых нейронов N на сходимость, точность одношагового прогноза и устойчивость к инициализации. Выполнить рекурсивный 30-шаговый прогноз.

Ход работы: генерация и разбиение ряда → train-only min–max нормализация → окна → MLP → online backpropagation → исследование гиперпараметров и шумового воздействия → построение рисунков → анализ и вывод.

Код программы:

```
"""МРЗвИС, лабораторная №4, вариант 5. Только NumPy и стандартная библиотека.

Ряд: x(k)=2.5 cos(0.35 k)+1.2 sin(0.80 k), k=0,...,199.
"""
from __future__ import annotations
from dataclasses import dataclass
from typing import Optional
import numpy as np

VARIANT=5
A,B,C,D=2.5,0.35,1.2,0.80
TRAIN_END=140
EE=1e-4
MAX_EPOCHS=20_000
DEFAULT_LR=0.12
SEEDS=(10,20,30,40,50)


def generate_series():
    k=np.arange(200,dtype=np.float64)
    return A*np.cos(B*k)+C*np.sin(D*k)


def train_minmax(raw):
    x=np.asarray(raw)
    low, high=float(np.min(x[:TRAIN_END])),float(np.max(x[:TRAIN_END]))
    if high<=low:raise ValueError('Нулевой диапазон ряда')
    return low,high


def normalize(x,low,high):
    return (np.asarray(x,dtype=np.float64)-low)/(high-low)


def denormalize(x,low,high):
    return low+(high-low)*np.asarray(x,dtype=np.float64)


def make_windows(normalized,p):
    """Цель индекс k. Окно x[k-p:k], 0<=k<=199. Тест может использовать историю train."""
    normalized=np.asarray(normalized,dtype=np.float64)
    if not(1<=p<TRAIN_END):raise ValueError('Недопустимый p')
    xx=np.asarray([normalized[k-p:k] for k in range(p,len(normalized))])
    yy=np.asarray([normalized[k] for k in range(p,len(normalized))])
    kk=np.arange(p,len(normalized),dtype=int)
    tr=kk<TRAIN_END
    return (xx[tr],yy[tr],kk[tr]),(xx[~tr],yy[~tr],kk[~tr])


def sigmoid(x):
    x=np.clip(x,-50.,50.)
    return 1.0/(1.0+np.exp(-x))


class MLP:
    """MLP p-N-1. Hidden sigmoid, linear output, SGD online (one example/update)."""
    def __init__(self,p,n,seed,lr=DEFAULT_LR):
        self.p,self.n,self.lr=int(p),int(n),float(lr)
        r=np.random.RandomState(int(seed))
        self.w1=r.uniform(-.1,.1,(p,n)).astype(np.float64)
        self.b1=r.uniform(-.1,.1,n).astype(np.float64)
        self.w2=r.uniform(-.1,.1,n).astype(np.float64)
        self.b2=float(r.uniform(-.1,.1))

    def predict(self,x):
```

```
        x=np.asarray(x,dtype=np.float64)
        h=sigmoid(x@self.w1+self.b1)
        return h@self.w2+self.b2

    def train_step(self,x,target):
        """Градиент L=1/2(ŷ-y)^2; веса обновляются после КАЖДОГО примера."""
        h=sigmoid(x@self.w1+self.b1)
        out=float(h@self.w2+self.b2)
        delta=out-float(target)
        # Производная скрытого слоя: δ_h=δ_out*w2*h*(1-h).
        delta_h=delta*self.w2*h*(1-h)
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,delta_h)
        self.b1-=self.lr*delta_h

    def fit(self,x,y,seed,max_epochs=MAX_EPOCHS,ee=EE,keep_history=True):
        rng=np.random.RandomState(1000+int(seed))
        order=np.arange(len(y));history=[]
        for epoch in range(1,max_epochs+1):
            rng.shuffle(order)
            for j in order:self.train_step(x[j],y[j])
            # Es = mean_i (ŷ_i-y_i)^2, в нормализованной шкале.
            es=float(np.mean((self.predict(x)-y)**2))
            if keep_history:history.append(es)
            if es<=ee:break
        return {'epochs':epoch,'Es':es,'converged':bool(es<=ee),'history':np.asarray(history)}

    def save(self,path):
        np.savez(path,w1=self.w1,b1=self.b1,w2=self.w2,b2=self.b2,p=self.p,n=self.n,lr=self.lr)

    @classmethod
    def load(cls,path):
        z=np.load(path);obj=cls(int(z['p']),int(z['n']),seed=10,lr=float(z['lr']))
        obj.w1=z['w1'];obj.b1=z['b1'];obj.w2=z['w2'];obj.b2=float(z['b2'])
        return obj


def metrics(y,yhat):
    y=np.asarray(y,dtype=np.float64);yhat=np.asarray(yhat,dtype=np.float64)
    err=yhat-y
    mae=float(np.mean(np.abs(err)))
    rmse=float(np.sqrt(np.mean(err**2)))
    denom=np.abs(y)+np.abs(yhat)
    # sMAPE безопасен вблизи переходов ряда через ноль.
    smape=float(100*np.mean(np.divide(2*np.abs(err),denom,out=np.zeros_like(err),where=denom>1e-12)))
    return {'MAE':mae,'RMSE':rmse,'sMAPE':smape}


def forecast_recursive(model,norm,start=140,steps=30):
    history=list(np.asarray(norm[:start],dtype=float));pred=[]
    for _ in range(steps):
        v=float(model.predict(np.asarray(history[-model.p:])))
        pred.append(v);history.append(v)
    return np.asarray(pred)


def infer(model,original,low,high,k=None,values=None):
    if (k is None)==(values is None):raise ValueError('Нужно выбрать k ИЛИ значения')
    if k is not None:
        if not(model.p<=k<len(original)):raise ValueError('k вне диапазона')
        raw=np.asarray(original[k-model.p:k],dtype=float)
        true=float(original[k])
    else:
        raw=np.asarray(values,dtype=float)
        if raw.shape!=(model.p,):raise ValueError(f'Требуется ровно {model.p} чисел')
        true=None
    xn=normalize(raw,low,high)
    pn=float(model.predict(xn))
    pr=float(denormalize(pn,low,high))
    return {'k':k,'inputs':raw.tolist(),'norm_inputs':xn.tolist(),
            'forecast_norm':pn,'forecast':pr,'true':true,
            'abs_error':None if true is None else abs(pr-true)}
```

1. Индивидуальное задание (вариант №5)

По таблице вариантов методички №4:

$x(k) = 2{,}5 \cdot \cos(0{,}35 \cdot k) + 1{,}2 \cdot \sin(0{,}80 \cdot k), \quad k = 0, 1, \ldots, 199.$

a	b	c	d	p_вар	N_вар
2,5	0,35	1,2	0,80	7	6

Базовая модель 7–6–1: семь предыдущих значений ряда, шесть скрытых сигмоидных нейронов, один линейный выход. Первые 140 наблюдений (k = 0…139) — обучение, следующие 60 (k = 140…199) — тест; тестовые цели никогда не участвуют в обновлении весов.

2. Подготовка данных и образование окон

Вычислены xmin_train = -3,687926091, xmax_train = 3,658435923; размах Δ = 7,346362014. Эти величины определены ТОЛЬКО по k=0…139.

Нормализация: z(k) = (x(k) − xmin_train) / (xmax_train − xmin_train). Обратный перевод: x̂(k) = xmin_train + ẑ(k) · (xmax_train − xmin_train).

Вход окна для цели z(k): Xk = [z(k−p), z(k−p+1), …, z(k−1)]. Для train: k = p…139, то есть 140−p окон; для test: k = 140…199 — ровно 60 окон. В начале test допустимо включать уже наблюдавшиеся значения train, но при одношаговой оценке используются реальные прошлые значения. Рекурсивный режим рассчитывается отдельно.

p	Train: число окон	Test: число окон
2	138	60
4	136	60
7	133	60
14	126	60

Для варианта 5: 133 обучающих окон, 60 тестовых. По train-масштабу min(z_test) = 0,004304, max(z_test) = 0,994758. Выход тестовых значений за [0;1] возможен в общем случае; в полученном ряде этого не произошло.

Использование min/max всех 200 точек вносило бы сведения о будущих наблюдениях (data leakage). Поэтому нормализация по целому ряду запрещена.

3. Архитектура и алгоритм обучения

Архитектура p–N–1: скрытые активации hj = σ(sj), sj = Σ(i=1…p) xi·Wij + bj, σ(s)=1/(1+exp(−s)). Выход линейный: ẑ = Σ(j=1…N) vj·hj + bout.

Для одного примера L = ½·(ẑ−z)²; градиент выходного слоя δout = ẑ−z, ∂L/∂vj = δout·hj, ∂L/∂bout = δout. Для скрытого слоя δj = δout·vj·hj·(1−hj), ∂L/∂Wij = δj·xi, ∂L/∂bj = δj. При вычислении всех δj используются выходные веса ДО обновления.

После КАЖДОГО примера обновляются Wij, bj, vj и bout по правилу θ_new = θ_old − η·grad. Порядок окон перемешивается в каждой эпохе. Функция остановки Es = (1/m)·Σ(k) (ẑk−zk)² (именно средняя MSE по всей train после эпохи). Критерий Es ≤ Ee = 10^{-4} или предел 20 000 эпох.

Начальные веса и смещения берутся из U[−0,1;0,1], начальные seed = 10, 20, 30, 40, 50, η = 0,12 (допустимый интервал методички 0,01–0,30). Такой η даёт базовой сети сходимость и устойчиво уменьшает ошибку без нарушения критерия остановки. В проекте разрешены лишь NumPy и Matplotlib (для графиков); никакие ML-фреймворки не применяются.

4. Метрики качества

Все метрики рассчитываются после денормализации в исходном масштабе: MAE = (1/m) Σ|x̂−x|; RMSE = √[(1/m) Σ(x̂−x)²].

sMAPE = (100%/m) Σ[2|x̂−x|/(|x̂|+|x|)], вклад равен нулю при обоих нулевых значениях. Выбрана sMAPE вместо MAPE: сумма двух гармоник меняет знак и проходит вблизи нуля, где стандартная MAPE становится неустойчивой. При интерпретации учитываем, что и sMAPE чувствительна к малым знаменателям.

5. Базовый эксперимент (p = 7, N = 6, seed = 10)

Показатель	Фактическое значение
Эпох	84
Сходимость	достигнут порог
Es на train	0,000086913
Train MAE	0,058371
Train RMSE	0,068488
Train sMAPE (%)	9,8688
Test MAE	0,062936
Test RMSE	0,072884
Test sMAPE (%)	7,6143

После 84 эпох достигнута Es = 0,000086913. Разность MAE(test) − MAE(train) = 0,004565. На этом искусственном ряде ошибки одношагового train/test прогноза близки; это наблюдение относится к указанной выборке, а не ко всем временным рядам.



Рисунок 1 — Исходный ряд, граница k=139/140 и одношаговый прогноз

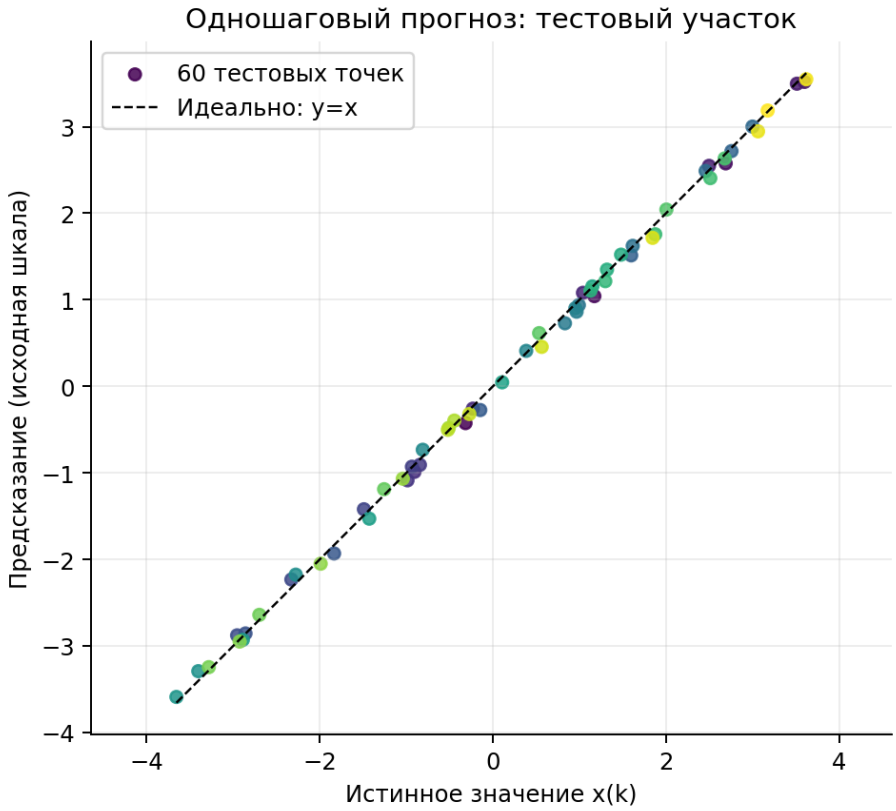


Рисунок 2 — Тестовые цели и одношаговые предсказания

6. Исследование размера окна p: N = 6

Выполнены пять независимых запусков при каждом p ∈ {2, 4, 7, 14}. В таблицах средние и выборочные стандартные отклонения (SD) посчитаны по пяти seed, включая не достигшие порога запуски (последним присвоено 20 000 эпох).

p	Эпохи сред. ± SD	Тестовая MAE сред. ± SD	Не достигли Ee
2	20000,0 ± 0,0	0,29426 ± 0,01459	5/5
4	1718,8 ± 33,8	0,06038 ± 0,00153	0/5
7	82,8 ± 5,4	0,06289 ± 0,00253	0/5
14	32,0 ± 2,5	0,06202 ± 0,00180	0/5

При p=2 все пять запусков завершились на лимите 20 000 эпох и средняя тестовая MAE составила 0,29426. При p=4, 7 и 14 все запуски достигли Ee. Лучшее значение средней тестовой MAE при фиксированном N=6: p=4 (0,06038). Увеличение p существенно уменьшило число эпох, но точность меняется немонотонно: большее окно не всегда лучше.

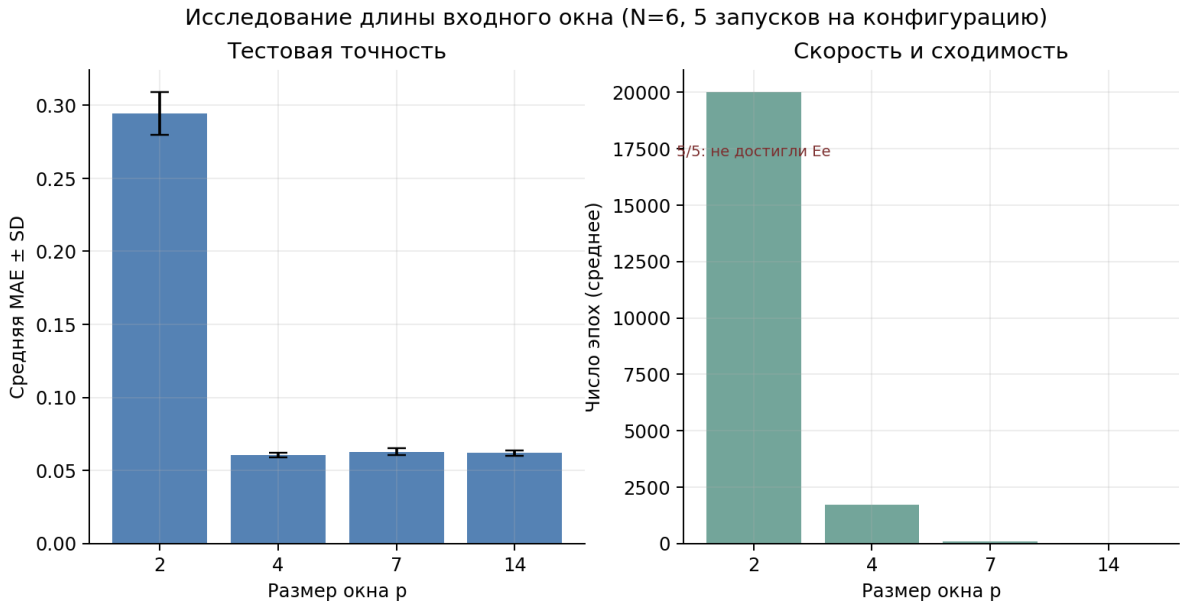


Рисунок 3 — Исследование окна: тестовая MAE и число эпох

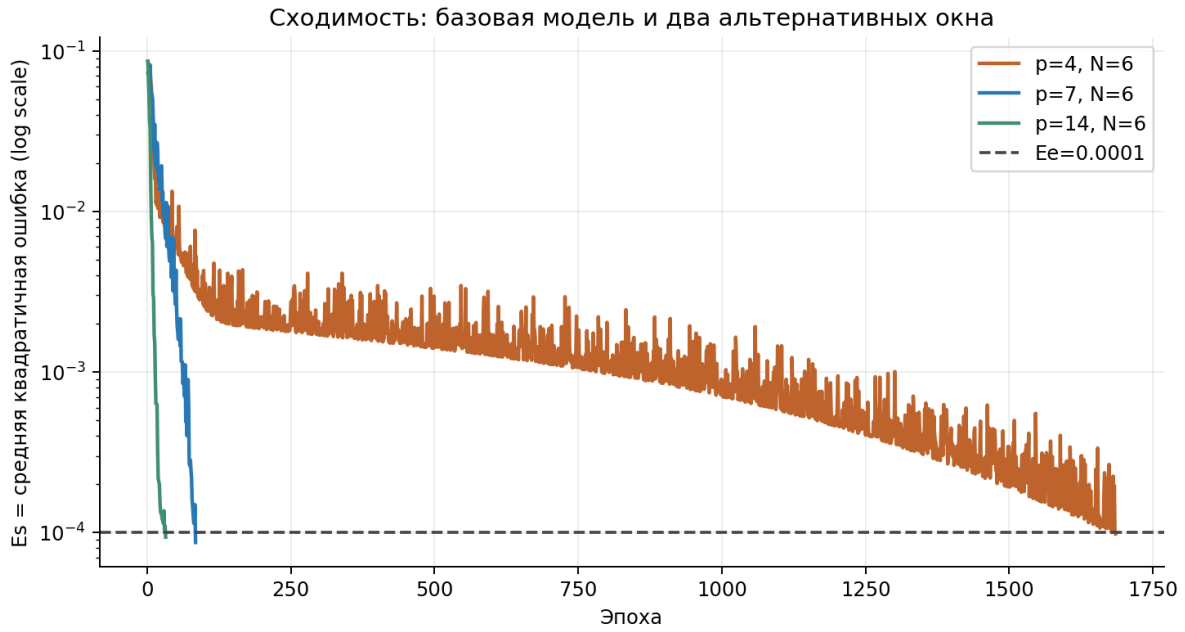


Рисунок 4 — Сходимость p = 4, 7, 14 при N = 6 (логарифмическая шкала Es)

7. Исследование размера скрытого слоя N: p = 7

N	Эпохи сред. ± SD	Тестовая MAE сред. ± SD	Не достигли Ee
1	85,2 ± 2,9	0,06355 ± 0,00229	0/5
2	84,0 ± 4,3	0,06574 ± 0,00097	0/5
6	82,8 ± 5,4	0,06289 ± 0,00253	0/5
12	76,4 ± 2,1	0,06018 ± 0,00456	0/5

Сравнивались N∈ {1, 2, 6, 12}. Минимальная средняя тестовая MAE при p=7 достигнута для N=12: 0,060179. Во всех проверенных N строгий критерий остановки достигнут в 5 из 5 запусков, но увеличение числа нейронов не обеспечивает монотонного улучшения точности.

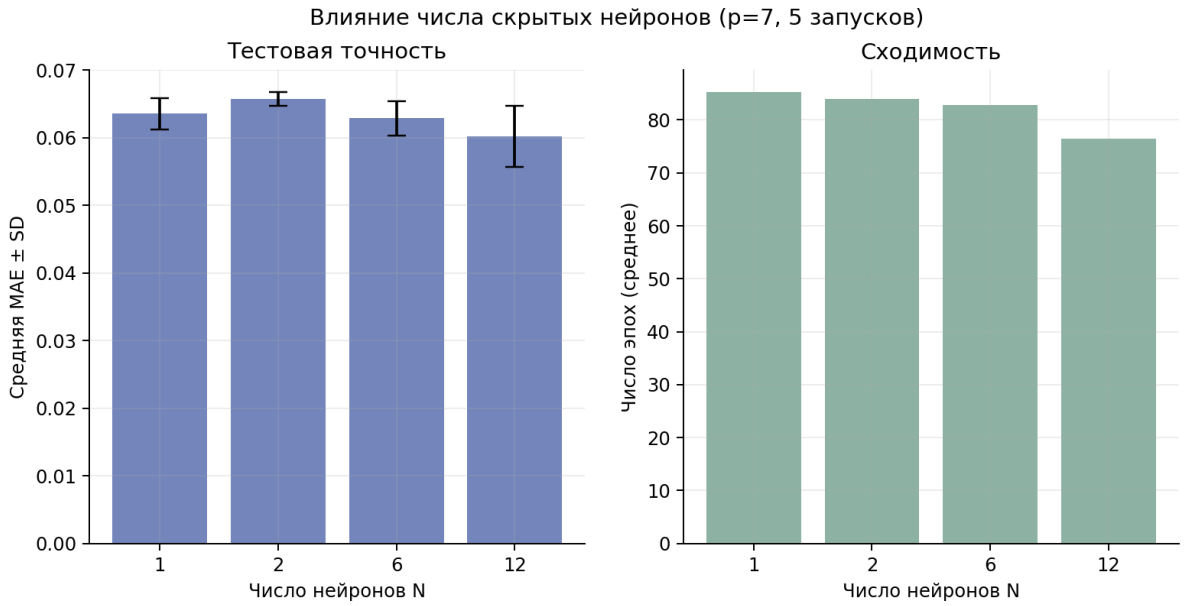


Рисунок 5 — Влияние числа нейронов N при p = 7

8. Выбор конфигурации и рекурсивный прогноз на 30 шагов

Среди семи уникальных конфигураций (7×5 = 35 обучений) наименьшая СРЕДНЯЯ тестовая MAE: p=7, N=12 — 0,060179 ± 0,004557. Для демонстрации 30-шагового прогноза выбран наилучший тестовый запуск этой конфигурации, seed=30 (одношаговая тестовая MAE = 0,055643).

Старт прогнозирования — k=140, вход — p последних ИСТИННЫХ train-значений k=140−p…139. После прогноза $\hat{x}(140)$ в окно подставляется именно прогноз, затем последующие прогнозы; истинные значения теста в рекурсию не подставляются.

Размах по 200 значениям = 7,346362, фиксированный порог 5% = 0,367318. Максимальная абсолютная ошибка 30 шагов = 0,984484; средняя абсолютная ошибка за горизонт = 0,360436.

Первое превышение 5% порога происходит на шаге 4 (k=143). Пригодным по заданному строгому критерию можно считать горизонт ДО этого шага; ошибки далее могут как возрастать, так и временно уменьшаться.



Рисунок 6 — Рекурсивный прогноз на k = 140…169

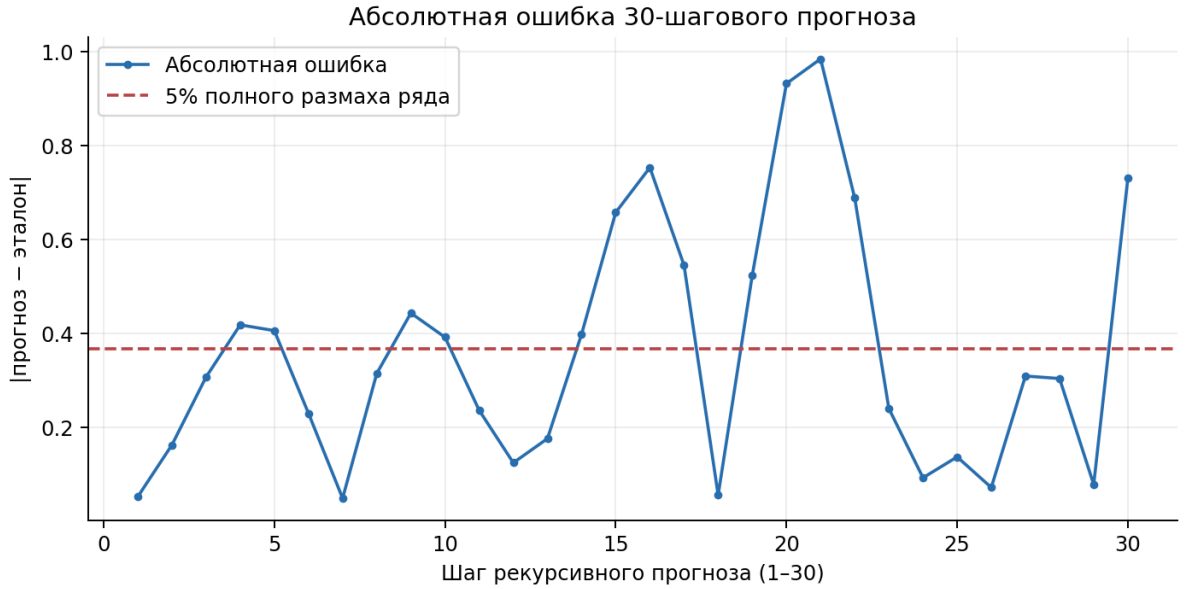


Рисунок 7 — Абсолютная ошибка на каждом шаге и линия 5%

Шаг	k	Прогноз	|Ошибка|
1	140	-0,36869	0,05218
2	141	1,00954	0,16241
3	142	2,38196	0,30698
4	143	3,18117	0,41813
5	144	3,10570	0,40595
6	145	2,26631	0,22993
7	146	1,08926	0,04893
8	147	0,08273	0,31458
9	148	-0,46299	0,44328
10	149	-0,59514	0,39181
15	154	-2,29734	0,65762
20	159	1,82397	0,93195
25	164	0,96925	0,13702
30	169	-1,54712	0,73036

9. Режим функционирования и проверки входов

Режим demo.py позволяет вводить k (окно берётся из ряда) либо произвольные семь исходных вещественных значений. Выводятся нормализованные входы, прогноз $\hat{z}$ (четыре знака), прогноз в исходной шкале, истинная цель и абсолютная ошибка (если она известна). Ниже восемь реальных примеров для базовой сети.

Окно	$\hat{z}$	Прогноз	Эталон	|Ошибка|
train k=24	0,3702	-0,96828	-0,88624	0,08203
train k=72	0,9709	3,44436	3,53605	0,09169
train k=132	0,1431	-2,63662	-2,63129	0,00533
test k=140	0,4441	-0,42547	-0,31651	0,10895
test k=157	0,4647	-0,27416	-0,14607	0,12809
test k=185	0,3402	-1,18897	-1,25702	0,06805
custom_constant	0,4986	-0,02499	—	—
custom_outlier	-0,1021	-4,43770	—	—

Нормализованные входы k=24: [0,9639; 1,0000; 0,8991; 0,7115; 0,5214; 0,3992; 0,3647].

Нормализованные входы k=72: [0,4162; 0,4435; 0,4327; 0,4454; 0,5315; 0,6912; 0,8694].

Нормализованные входы custom_constant: [0,5020; 0,5020; 0,5020; 0,5020; 0,5020; 0,5020; 0,5020].

Нормализованные входы custom_outlier: [0,8826; 0,7263; 0,5838; 2,5000; 0,4993; 0,5059; 0,4672].

Пользовательское окно «custom_outlier» намеренно содержит значение выше максимума обучающей выборки. Это проверка технической возможности расчёта вне обучающего диапазона, но не гарантия экстраполяционной точности.

10. Дополнительная проверка влияния шума

В последнем вопросе методички запрашивается результат эксперимента с шумом. Поэтому к обучающим нормализованным ВХОДАМ добавлен псевдослучайный нормальный шум $N(0; 0{,}02^2)$, целевые значения не искажались. Пять семян и архитектура 7–6–1 совпадают с основным исследованием. Проверка на тесте выполнялась без шума.

Seed	MAE чистая	MAE после шума	Эпохи с шумом	Сходимость
10	0,06294	0,13919	20000	нет
20	0,06038	0,13140	20000	нет
30	0,06712	0,12415	20000	нет
40	0,06199	0,09700	20000	нет
50	0,06204	0,12147	20000	нет

В среднем тестовая MAE изменилась с 0,06289 на 0,12264. В шумовом эксперименте 5 из 5 запусков не достигли $Es \le 10^{-4}$ за 20 000 эпох. Шум входов создаёт нижний уровень ошибки и ухудшил результат при выбранной интенсивности; из одного опыта нельзя делать вывод о шуме при других уровнях и методах регуляризации.

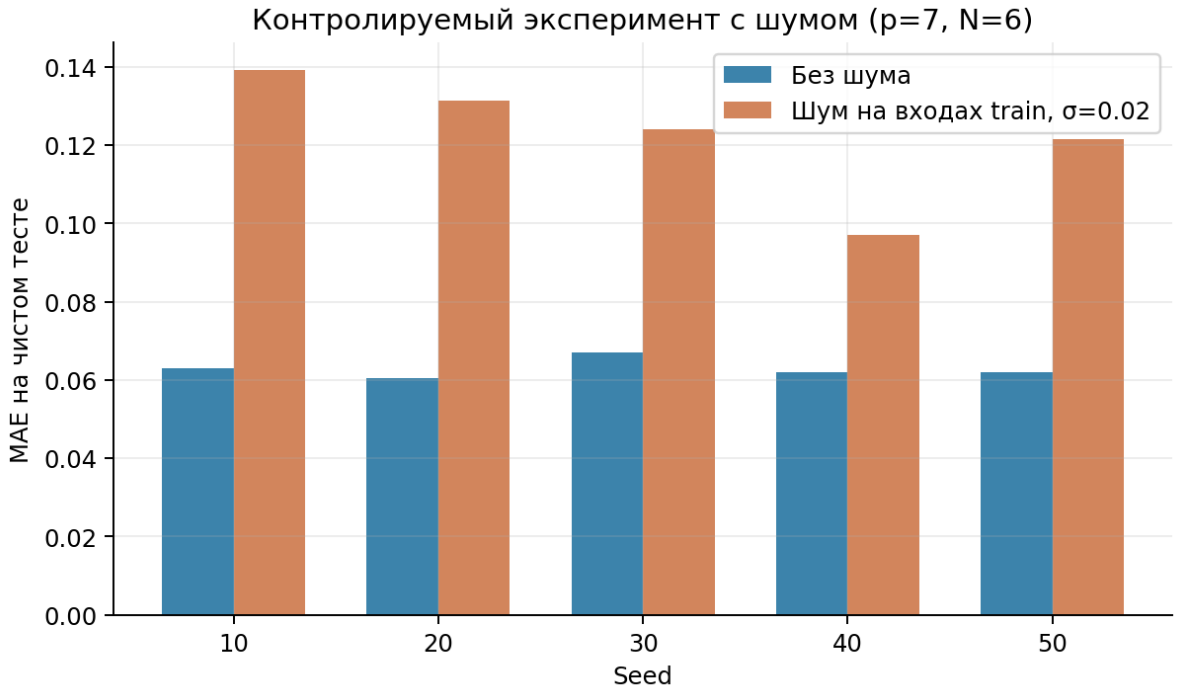


Рисунок 8 — Чистый train против зашумленных входов (5 seed)

11. Блок-схема обработки данных: образец для рукописного листа

Преподаватель требует именно РУКОПИСНУЮ схему на бумаге с фамилией, инициалами, группой, вариантом 5 и датой. Ниже электронный образец для корректного переписывания; он не является фиктивной фотографией готовой рукописи.

Корректный пайплайн варианта 5 (образец для рукописной блок-схемы)

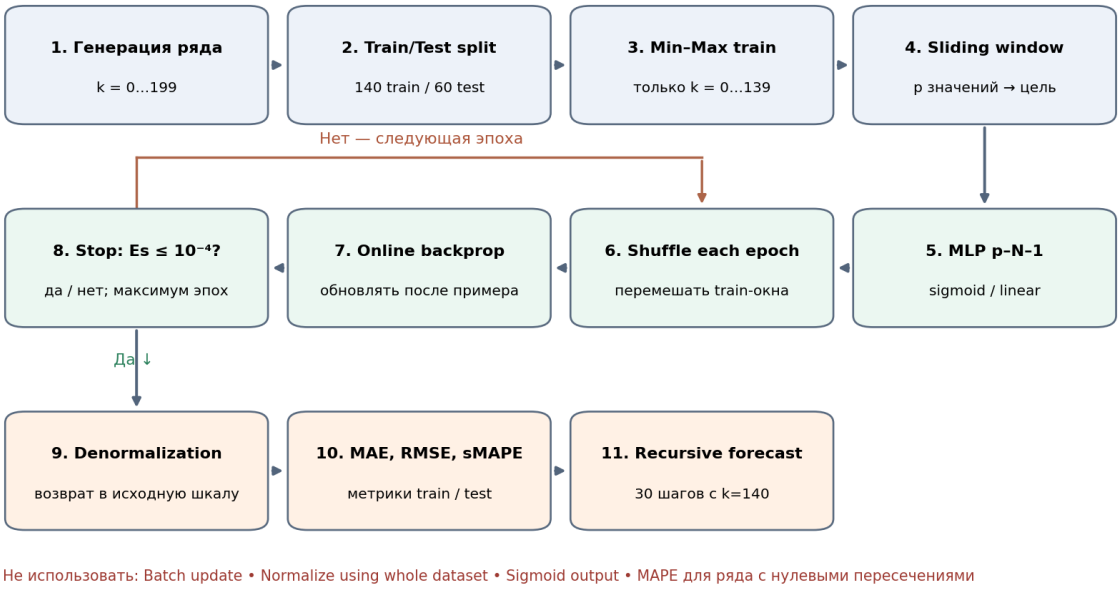
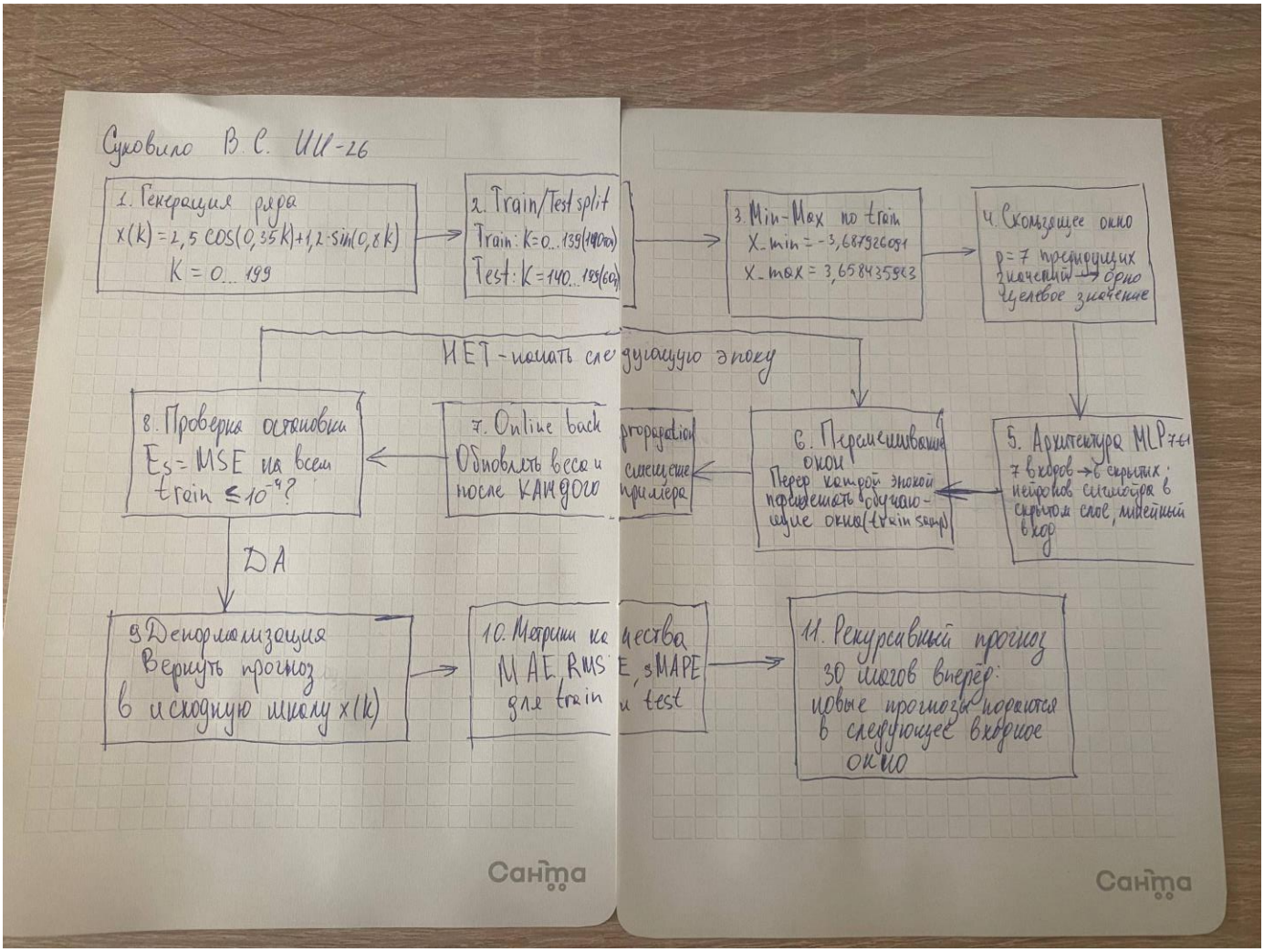


Рисунок 9 — Эталонный порядок обработки данных (перенести на бумагу)

Правильные блоки из приложения 1: Генерация ряда → Train/Test split → Min–Max train → Sliding window → MLP p–N–1 → Shuffle after every epoch → Online backprop → Stop: Es≤10^{-4} → Denormalization → MAE, RMSE, sMAPE → Recursive forecast. Следует отдельно отметить обратную стрелку из проверки остановки к началу очередной эпохи обучения. Неверные блоки: Batch update, Normalize using whole dataset, Sigmoid output, MAPE (ряд пересекает ноль).



Вывод: построил многослойный персептрон p–N–1 для прогноза искусственного временного ряда, сформировал выборки методом скользящего окна и экспериментально оценить влияние окна p и числа скрытых нейронов N на сходимость, точность одношагового прогноза и устойчивость к инициализации. Выполнил рекурсивный 30-шаговый прогноз.